

Guide de setup Raspberry Pi pour Cycle Analyst App

Objectif

Ce guide decrit un setup reproductible pour installer le repo sur plusieurs Raspberry Pi, un par vehicule, en partant d'une carte SD vierge.

Objectifs du setup :

- chaque Pi a un nom unique sur le reseau local
- l'app demarre automatiquement au boot
- l'interface web est accessible depuis un navigateur sans taper l'IP
- l'URL reste la meme sur le hotspot du Pi et sur un autre reseau local, par exemple le hotspot du telephone, tant que le reseau laisse passer le mDNS

Exemple de convention :

- vehicule 1 -> `sc-vehicule-1.local`
- vehicule 2 -> `sc-vehicule-2.local`
- vehicule 3 -> `sc-vehicule-3.local`

Hypotheses

- Raspberry Pi OS 64-bit
- acces SSH active
- utilisateur Linux commun sur tous les Pi, par exemple `jeandard`
- repo installe dans `/home/jeandard/Cycle-Analyst-App`
- l'app Flask principale tourne sur le port `5050`
- on veut exposer l'app sur le port `80` via `nginx` pour acceder a `http://sc-vehicule-1.local` sans saisir `:5050`

Vue d'ensemble

Le setup final repose sur 4 briques :

- un hostname unique par Pi
- `avahi-daemon` pour publier `*.local` en mDNS
- un service `systemd` pour lancer l'app au boot
- `nginx` pour faire le proxy de `127.0.0.1:5050` vers le port `80`

Etape 1 - Preparer la carte SD

Avec Raspberry Pi Imager

1. Insérer la carte SD dans le Mac.
2. Ouvrir Raspberry Pi Imager.
3. Choisir le modèle de Pi si l'outil le demande.
4. Choisir l'OS :
 - recommande : `Raspberry Pi OS Lite (64-bit)` si le Pi n'a pas besoin d'interface graphique locale
 - acceptable : `Raspberry Pi OS (64-bit)` si tu veux aussi un bureau local
5. Choisir la carte SD.
6. Cliquer sur les options avancées avant l'écriture.

Paramètres conseillés dans les options avancées

- définir un hostname unique des l'image SD :
 - Pi 1 -> `sc-vehicule-1`
 - Pi 2 -> `sc-vehicule-2`
 - Pi 3 -> `sc-vehicule-3`
- activer SSH
- définir le nom d'utilisateur, par exemple `jeandard`
- définir le mot de passe
- si tu as déjà une clé publique SSH, l'ajouter directement
- régler le pays Wi-Fi
- régler la locale et le fuseau horaire
- si tu connais déjà le SSID et le mot de passe du hotspot du téléphone, tu peux aussi les préconfigurer

Remarque importante

Si tu définis déjà le hostname dans Raspberry Pi Imager, tu t'évites une étape ensuite. Si un Pi a déjà été booté avec le nom `cycLe`, il faudra corriger le hostname plus tard.

Etape 2 - Premier boot

1. Insérer la carte SD dans le Pi.
2. Brancher l'alimentation.
3. Attendre le premier boot.
4. Connecter le Pi à un réseau :

- soit le hotspot du telephone
- soit le hotspot Wi-Fi cree par le Pi
- soit un autre reseau local de test

5. Depuis le Mac, tester l'acces SSH :

```
ssh jeandard@sc-vehicule-1.local
```

Si le nom `.local` ne repond pas au premier boot, utiliser l'IP pour terminer la config initiale :

```
ssh jeandard@IP_DU_PI
```

Pour trouver l'IP depuis le Pi :

```
hostname -I
```

Etape 3 - Verification de base

Une fois connecte en SSH :

```
hostname
hostnamectl
cat /etc/hostname
hostname -I
```

Le resultat attendu pour le Pi 1 :

- `hostname -> sc-vehicule-1`
- `hostnamectl -> Static hostname: sc-vehicule-1`
- `/etc/hostname -> sc-vehicule-1`

Etape 4 - Corriger ou changer le hostname si besoin

Si le Pi porte encore un ancien nom, par exemple `cycle` :

```
sudo hostnamectl set-hostname sc-vehicule-1
echo 'preserve_hostname: true' | sudo tee /etc/cloud/cloud.cfg.d/99-preserve-hostname.cfg
sudo reboot
```

Apres reboot, verifier a nouveau :

```
hostname
hostnamectl
cat /etc/hostname
```

Faire la meme chose sur les autres Pi en adaptant le nom.

Etape 5 - Mise a jour systeme et paquets utiles

Sur chaque Pi :

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git python3 python3-venv python3-pip nginx avahi-daemon
```

Puis activer `avahi-daemon` :

```
sudo systemctl enable avahi-daemon
sudo systemctl start avahi-daemon
systemctl status avahi-daemon --no-pager
```

Etape 6 - Installer le repo

Depuis le home de l'utilisateur :

```
cd /home/jeandard
git clone <URL_DU_REPO> Cycle-Analyst-App
cd /home/jeandard/Cycle-Analyst-App
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Remplace `<URL_DU_REPO>` par l'URL Git du projet.

Etape 7 - Tester l'app manuellement

Toujours depuis le repo :

```
cd /home/jeandard/Cycle-Analyst-App
source .venv/bin/activate
python cycle_server.py
```

Le repo expose l'app sur `0.0.0.0:5050`, donc l'accès brut est :

- localement sur le Pi : `http://127.0.0.1:5050`
- depuis un autre appareil du même réseau : `http://IP_DU_PI:5050`

Pour vérifier depuis le Pi :

```
curl http://127.0.0.1:5050
```

Arrêter ensuite le processus avec `Ctrl+C`.

Etape 8 - Créer le service systemd

Créer un service pour démarrer l'app au boot :

```
sudo nano /etc/systemd/system/cycle-analyst.service
```

Contenu :

```
[Unit]
Description=Cycle Analyst App
After=network-online.target
Wants=network-online.target

[Service]
User=jeandard
WorkingDirectory=/home/jeandard/Cycle-Analyst-App
Environment=PATH=/home/jeandard/Cycle-Analyst-App/.venv/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin
ExecStart=/home/jeandard/Cycle-Analyst-App/.venv/bin/python /home/jeandard/Cycle-Analyst-App/cycle_serv
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Activer le service :

```
sudo systemctl daemon-reload
sudo systemctl enable cycle-analyst.service
sudo systemctl start cycle-analyst.service
sudo systemctl status cycle-analyst.service --no-pager
```

Vérifier ensuite :

```
curl http://127.0.0.1:5050
```

Etape 9 - Exposer l'app avec une URL locale sans l'IP

9.1 Vérifier le mDNS .local

Avec avahi-daemon, le Pi doit être résolvable comme :

- sc-vehicule-1.local
- sc-vehicule-2.local

Vérification sur le Pi :

```
avahi-resolve-host-name sc-vehicule-1.local
```

Vérification depuis le Mac :

```
ping sc-vehicule-1.local
ssh jeandard@sc-vehicule-1.local
```

9.2 Configurer nginx

Créer le fichier de site :

```
sudo nano /etc/nginx/sites-available/cycle-analyst
```

Contenu pour le Pi 1 :

```
server {
```

```

listen 80;
server_name sc-vehicule-1.local;

location / {
    proxy_pass http://127.0.0.1:5050;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}

```

Contenu pour le Pi 2 :

```

server {
    listen 80;
    server_name sc-vehicule-2.local;

    location / {
        proxy_pass http://127.0.0.1:5050;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

Activer le site :

```

sudo ln -s /etc/nginx/sites-available/cycle-analyst /etc/nginx/sites-enabled/cycle-analyst
sudo nginx -t
sudo systemctl reload nginx

```

Vérification :

```
curl http://127.0.0.1
```

Depuis le téléphone ou le Mac :

- `http://sc-vehicule-1.local`
- `http://sc-vehicule-2.local`

Etape 10 - Config SSH pour VS Code

Sur le Mac, éditer `~/.ssh/config` :

```

Host sc1 sc-vehicule-1.local
    HostName sc-vehicule-1.local
    User jeandard
    Port 22
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes
    ServerAliveInterval 30
    ServerAliveCountMax 6

Host sc2 sc-vehicule-2.local
    HostName sc-vehicule-2.local
    User jeandard
    Port 22
    IdentityFile ~/.ssh/id_ed25519

```

```
IdentitiesOnly yes
ServerAliveInterval 30
ServerAliveCountMax 6
```

Ensuite, dans VS Code :

- Remote-SSH: Connect to Host...
- choisir `sc1` ou `sc2`

Etape 11 - Option monitor multi-vehicules

Si tu utilises aussi le `monitor_server`, il peut etre utile de fixer explicitement un identifiant unique par Pi plutot que de dependre uniquement du hostname.

Exemple, dans le service `systemd`, ajouter :

```
Environment=MONITOR_DEVICE_ID=sc-vehicule-1
```

et sur un autre Pi :

```
Environment=MONITOR_DEVICE_ID=sc-vehicule-2
```

Ce n'est pas obligatoire pour l'accès URL, mais c'est utile si plusieurs Pi uploadent vers le meme monitor.

Etape 12 - Cas du hotspot du Pi et du hotspot du telephone

Ce qui marche bien

Le meilleur nom local hors ligne est :

- `sc-vehicule-1.local`
- `sc-vehicule-2.local`

Ce nom peut marcher :

- sur le hotspot du Pi
- sur le hotspot du telephone
- sur un autre reseau Wi-Fi local

La limite importante

Le suffixe `.local` repose sur le mDNS. Certains hotspots de telephone filtrent :

- le multicast
- la communication entre clients Wi-Fi

Dans ce cas :

- SSH par IP peut marcher
- le navigateur via IP peut marcher
- `sc-vehicule-1.local` peut ne pas se resoudre

Conclusion pratique :

- si le hotspot du telephone laisse passer le mDNS, tu gardes la meme URL
- sinon, il faut passer par l'IP sur ce reseau-la
- si tu veux un comportement garanti hors ligne avec un nom de domaine stable, il faut controler le DNS du reseau, donc plutot utiliser le hotspot du Pi

Etape 13 - Strategie pour deployer plusieurs Pi

Option A - Une SD preparee individuellement par vehicule

Avantages :

- simple
- propre
- chaque Pi a son hostname des le premier boot
- evite les collisions de cle SSH

Procedure :

1. preparer chaque SD avec Raspberry Pi Imager
2. definir le hostname correct avant le premier boot
3. installer le repo
4. activer `systemd`, `avahi`, `nginx`

Option B - Cloner une SD deja configuree

Possible, mais plus fragile.

Apres clonage, sur chaque Pi clone, verifier au minimum :

- hostname unique
- cle SSH hote unique
- eventuel `MONITOR_DEVICE_ID` unique

Pour verifier l'empreinte SSH hote :

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```


Si deux Pi clones ont la meme empreinte, regenerer les clefs SSH sur l'un d'eux :

```
sudo rm /etc/ssh/ssh_host_*
sudo dpkg-reconfigure openssh-server
sudo systemctl restart ssh
```

Checklist finale par vehicule

- hostname correct
- acces SSH OK
- avahi-daemon actif
- repo clone dans /home/jeandard/Cycle-Analyst-App
- venv Python cree
- dependances installees
- cycle-analyst.service actif
- nginx actif
- http://127.0.0.1 repond sur le Pi
- http://sc-vehicule-X.local repond depuis un autre appareil du meme reseau

Depannage

Le nom .local ne marche pas

Verifier :

```
systemctl status avahi-daemon --no-pager
hostname
hostnamectl
hostname -I
```

Tester depuis le Mac :

```
ping sc-vehicule-1.local
ssh jeandard@sc-vehicule-1.local
```

Si l'IP marche mais pas le nom .local, le reseau bloque probablement le mDNS.

VS Code se connecte en SSH mais pas en Remote-SSH

Si le SSH simple marche mais pas VS Code, supprimer l'ancien serveur distant :

```
rm -rf ~/.vscode-server ~/.vscode-remote
```

Puis retenter la connexion depuis VS Code.

Changement de cle SSH dans known_hosts

Si le Mac affiche `REMOTE HOST IDENTIFICATION HAS CHANGED`, nettoyer l'entree :

```
ssh-keygen -R sc-vehicule-1.local  
ssh-keygen -R sc-vehicule-2.local  
ssh-keygen -R cycle.local
```

Resume minimal a reutiliser pour chaque nouveau Pi

1. preparer la SD avec un hostname unique et SSH actif
2. booter le Pi et verifier le hostname
3. installer `git`, `python3-venv`, `nginx`, `avahi-daemon`
4. cloner le repo et installer les dependances Python
5. tester `python cycle_server.py`
6. creer le service `systemd`
7. creer la conf `nginx`
8. verifier `http://sc-vehicule-X.local`

Fichiers et commandes utiles

- service app : `/etc/systemd/system/cycle-analyst.service`
- site nginx : `/etc/nginx/sites-available/cycle-analyst`
- config SSH Mac : `~/.ssh/config`
- verifier le service app :

```
sudo systemctl status cycle-analyst.service --no-pager
```

- verifier nginx :

```
sudo nginx -t  
sudo systemctl status nginx --no-pager
```

- verifier avahi :

```
systemctl status avahi-daemon --no-pager
```